



INDIA.RUNS

Build what next India runs on

Team Name: TalentLens AI

Team Leader: Mohd Ibadullah

Problem Statement: Intelligent Candidate Discovery & Ranking (100K candidates on CPU)

Solution Overview

- What is your proposed solution?

TalentLens AI is a 6-stage intelligent candidate ranking pipeline that processes 100,000 profiles in ~28 seconds on CPU-only hardware. It combines BM25 filtering, Honeypot detection, BGE semantic embeddings (BAAI/bge-base-en-v1.5), Cross-Encoder reranking (ms-marco-MiniLM-L6-v2), and Redrob behavioral signals to produce a validated ranked shortlist of the top 100 candidates.

What differentiates your approach from traditional candidate matching systems?

- Cross-Encoder Reranking: Applies ms-marco-MiniLM-L6-v2 on top-150 candidates for precise lexical-semantic contextual relevance.
- Honeypot Trap Detector: Identifies keyword-stuffed decoy profiles by cross-checking career history against claimed skills (0.0% honeypot rate in top 100).
- Signal-Aware Scoring: Integrates recruiter response rate, notice period, GitHub activity, and profile completeness — treating missing values as neutral, not penalties.
- Hybrid Reasoning: Uses a deterministic rule-based generator for offline reproduction, and an offline Gemini/Groq utility for natural, rank-consistent CSV explanations.

JD Understanding & Candidate Evaluation

- What are the key requirements extracted from the JD?

- Role title, seniority level, minimum years of experience
- Required skills: Python, NLP, Vector Search, Embeddings, LLM Fine-tuning, Information Retrieval, Ranking Systems
- Nice-to-have skills: PyTorch, TensorFlow, MLOps, RAG, FAISS, Milvus, Pinecone
- Domain keywords for lexical search boosting

Which candidate signals are most important for determining relevance?

- Cross-Encoder reranking relevance (ms-marco-MiniLM-L6-v2)
- Semantic similarity (BAAI/bge-base-en-v1.5, 768-dim embeddings)
- Skill overlap with proficiency weighting and endorsement bonuses
- Title relevance + years of experience fit
- Redrob behavioral signals (recruiter response rate, notice period, GitHub activity, profile completeness)
- Career history consistency (used for honeypot detection)

Ranking Methodology

- How does your system retrieve, score, and rank candidates?

Stage 1 — BM25 Filter: Narrows 100K to 1,000 candidates in ~8 seconds using tokenized JD terms.

Stage 2 — Honeypot Trap Detector: Flags decoys with career-title mismatches and boilerplate summaries.

Stage 3 — Semantic Embedding: BAAI/bge-base-en-v1.5 computes similarity between JD and candidates.

Stage 4 — Cross-Encoder Reranking: Reranks top 150 candidates using ms-marco-MiniLM-L6-v2.

Stage 5 — Weighted Feature Scoring: Final Score = $100 \times (0.40 \times \text{Semantic} + 0.20 \times \text{Skills} + 0.20 \times \text{Title} + 0.10 \times \text{Signals}) / 0.90 - 0.40 \times \text{TrapPenalty}$.

Stage 6 — Tie-breaking: score descending, then candidate_id ascending.

Models & Algorithms Used:

- BM25Okapi (rank-bm25) for lexical retrieval
- BAAI/bge-base-en-v1.5 transformer (768-dim, CLS pooling, instruction prefix)
- cross-encoder/ms-marco-MiniLM-L6-v2 for contextual reranking
- RapidFuzz token_sort_ratio (85% threshold) for fuzzy skill matching
- Rule-based honeypot detector with 45 keywords across 12 categories

Explainability & Data Validation

- How are ranking decisions explained?

- Each candidate gets a unique reasoning referencing their actual matched skills, YoE, title, and signal data.
- Tone is rank-consistent: top-15 get strong endorsements, 50+ get honest gap acknowledgments.

How do you prevent hallucinations or unsupported justifications?

- Offline reproduction script runs fully offline with deterministic rule-based reasoning.
- Offline Gemini/Groq generator utility creates natural, rank-consistent reasons in the final CSV.
- Skills mentioned in reasoning are pulled directly from the candidate's profile data, never fabricated.

How does your solution handle inconsistent or suspicious profiles?

- Honeypot detector checks: summary boilerplate, AI skill-stuffing on non-AI titles, career mismatches.
- Trap penalty of up to 40 points subtracted from final score.
- Result: 0.0% honeypot rate in top 100.

End-to-End Workflow

- Complete Workflow (JD Input → Ranked CSV Output):

1. JD Parsing → Extracts role title, required/nice-to-have skills, experience, seniority, domain keywords.
2. BM25 Filter → 100K candidates streamed from JSONL, lexically filtered to top 1,000.
3. Honeypot Detection → Candidates checked for career-title inconsistency and template summaries.
4. Semantic Embedding → BGE encodes JD and candidate profiles. Cosine similarity computed.
5. Cross-Encoder → MiniLM reranks the top 150 candidates.
6. Feature Scoring → Weighted: Semantic(40%) + Skills(20%) + Title(20%) + Signals(10%) – Trap(40%).
7. Reasoning Generation → Rank-aware, candidate-specific reasons written to CSV.

Single reproduce command:

```
python src/run_pipeline_full.py --candidates ./candidates.jsonl --out ./outputs/mohd_ibadullah.csv
```

Runtime: ~28 seconds | Memory: ~1.3 GB | CPU only | No network calls

System Architecture

Results & Performance

- Ranking Quality Metrics (vs BM25 Lexical Baseline):

- Precision@10: 1.0000 (ours) vs 0.4000 (baseline) — +150% boost
- Recall@20: 1.0000 (ours) vs 0.4000 (baseline) — +150% boost
- NDCG@10: 1.0000 (ours) vs 0.4288 (baseline) — +133.2% boost
- Honeypot rate: 0.0% in Top 1000 after Stage 2 filter (vs 30.1% baseline)
- 100 unique, non-templated reasonings, scores monotonically non-increasing

Runtime & Compute Constraints (reproduced locally on CPU):

- Total runtime: ~28 seconds (budget: 300 seconds) — 90% under limit
- Memory: ~1.3 GB (budget: 16 GB) — 91% under limit
- CPU only — no GPU used
- No external API calls during ranking (fully sandboxed)

Technologies Used

- Core Technologies & Why:

- Python 3.12 — Primary programming language
- BAAI/bge-base-en-v1.5 — 768-dim semantic embeddings, superior retrieval quality
- cross-encoder/ms-marco-MiniLM-L6-v2 — Fine-grained semantic reranking on top 150 candidates
- PyTorch — Model inference backend (CPU mode)
- rank-bm25 (BM25Okapi) — Lexical filtering, fast and memory-efficient
- RapidFuzz — Fuzzy skill matching with token_sort_ratio
- Pandas & NumPy — Numerical operations and output handling
- Streamlit — Interactive web dashboard with dark theme and weight sliders
- Gemini 2.5 Flash / Groq APIs — Interview question generator (demo feature)

Submission Assets

GitHub Repository (Public):

<https://github.com/mohd-ibadullah/TalentLensAI>

Live Demo (Streamlit Cloud):

<https://talentlensai-nxrk7zxjmaxvnwnubyz7n.streamlit.app>

Ranked Output File:

outputs/mohd_ibadullah.csv (100 candidates, officially validated)

Reproduce Command:

```
python src/run_pipeline_full.py --candidates ./candidates.jsonl --out ./outputs/mohd_ibadullah.csv
```

Note: Live Demo runs on 50-candidate sample. Full 100K requires local setup.



INDIA.RUNS

Build what next India runs on

THANK YOU